

VIT BHOPAL UNIVERSITY

VITYarthi – Your Learning Destination

PROJECT INFORMATION

Project title	Shopping and billing system
Course title	Introduction To Problem Solving And programming
Course code	CSE 1021
Course type	Flipped course
Course credits	4
Professor	Dr. Manoj Kumar
Slot	B11+B12+B13+C14+E11+E12
Submitted by	Meghan Umale
Registration number	26BAI10922
College	VIT Bhopal University
Program	B.Tech CSE AI/ML
Year / Semester	1st Year, 1st Semester
Submission date	30/09/26

INTRODUCTION :

Every shop, from a small neighbourhood store to a large supermarket, has to do the same job at the counter: identify what the customer is buying, work out the cost of each item, apply any offers, add tax and produce a correct bill. When this is done by hand with a calculator and a notebook, mistakes in multiplication, discount or tax are common and the process is slow.

This project is a small-scale simulation of such a billing counter, written in Python with a menu-driven console interface. The user views a fixed product list, adds items with a quantity to a cart, reviews the cart, and finally generates a bill. The bill applies a quantity-based discount slab and 5% GST automatically and prints the final amount payable. Control returns to the menu after each operation.

The application is intentionally lightweight. It uses only core Python features (functions, dictionaries, lists, loops and conditional statements) and needs no external library, which keeps it easy to install, run, review and grade as a first-year academic project.

Key characteristics of the system

- Menu-driven interaction
- Dictionary-based product catalogue and list-based cart
- Automatic discount slabs and GST calculation
- Function-per-operation code structure
- Clear messages for an empty cart or an invalid choice

PROBLEM STATEMENT :

Manual billing at a shop is slow and error-prone. Prices may be recalled incorrectly, quantities multiplied wrongly, discounts applied inconsistently and tax forgotten. A small structured program can remove these problems by keeping the product prices in one place and applying the same rules to every bill.

The proposed system shows a product catalogue, lets the user add products with quantities to a cart, shows the running cart with a subtotal, and generates a final bill with discount and GST. The project focuses on a clearly defined academic scope rather than attempting to reproduce a full point-of-sale system: the goal is correctness and clarity of implementation rather than breadth of features.

PROJECT OBJECTIVES :

The main objective is to develop an understandable Python program that models a basic shopping and billing workflow. Additional objectives are to apply programming concepts in a complete project workflow, from requirements through design, implementation and testing.

- Apply core Python concepts: functions, dictionaries, lists, loops and conditionals.
- Store a product catalogue of ten items with names and prices.
- Let the user add products with quantities to a cart and view the cart with a subtotal.
- Calculate a discount of 5% (subtotal of Rs. 500 or more) or 10% (subtotal of Rs. 1000 or more).
- Calculate 5% GST on the discounted amount and print the final payable amount.
- Provide a simple menu that does not crash on an invalid menu choice.
- Test the program with a documented sequence of inputs and record the results.

Together, these objectives ensure that the finished project is not just a working script, but a small, well-organised piece of software that follows recognisable engineering practice.

SCOPE OF THE PROJECT :

The current project covers a single-user, single-session shopping and billing simulation through a command-line interface. It focuses on browsing a fixed catalogue, building a cart and generating one bill, using only standard Python features.

Editing the catalogue at run time, removing cart items, customer accounts, permanent storage, payment processing and graphical interfaces are outside the current scope. They are listed under future enhancements because they need infrastructure beyond what a single-semester course project is expected to deliver.

- In scope: product display, add to cart, view cart, bill generation
- In scope: discount slabs and GST calculation
- In scope: terminal interface
- Out of scope: persistent storage (files or database)
- Out of scope: payment, customer login, stock management
- Out of scope: GUI or web interface

FUNCTIONAL REQUIREMENTS :

The functional requirements define the operations available to the user through the terminal menu.

- Display the product catalogue with product number, name and price.
- Add a product to the cart by product number and quantity.
- Report 'Invalid product number.' for a number that is not in the catalogue.
- Show every cart line with price, quantity and line total, plus the subtotal.
- Report 'Your cart is empty.' when the cart has no items.
- Generate a bill listing each item, the subtotal, discount, GST and final amount.
- Apply a 5% discount for a subtotal of Rs. 500 or more and 10% for Rs. 1000 or more.
- Charge 5% GST on the amount remaining after discount.
- Refuse to generate a bill for an empty cart.
- Handle invalid menu choices without terminating the program.
- Exit cleanly with a thank-you message.

NON-FUNCTIONAL REQUIREMENTS :

Non-functional requirements describe qualities expected from the system beyond the specific operations it performs.

- Usability: simple numbered menu and clear prompts.
- Accuracy: line totals, discount and GST are calculated by the program, never typed by the user.
- Maintainability: one function per operation, so each can be read and changed separately.
- Performance: instant response for the small cart held in memory.
- Resource efficiency: no external libraries required.
- Portability: runs on any system with a standard Python 3 interpreter.
- Testability: operations can be checked repeatedly with a fixed input sequence.

REQUIREMENTS TRACEABILITY MATRIX :

The table below maps each functional requirement to the function primarily responsible for implementing it, and to the test case that verifies it. This traceability view helps confirm that every requirement is both implemented and tested.

Req. ID	Requirement	Responsible function	Verified by
FR-1	Display product catalogue	display_products()	Display products test
FR-2	Add product with quantity to cart	add_to_cart()	Add product test
FR-3	Reject unknown product number	add_to_cart()	Invalid product test
FR-4	Show cart lines and subtotal	show_cart()	View cart test
FR-5	Report an empty cart	show_cart()	View empty cart test
FR-6	Generate itemised bill	generate_bill()	Bill tests
FR-7	Apply 5% / 10% discount slabs	generate_bill()	5% and 10% discount tests
FR-8	Add 5% GST after discount	generate_bill()	Bill tests
FR-9	Refuse bill for empty cart	generate_bill()	Bill on empty cart test
FR-10	Handle invalid menu choice	Main menu loop	Invalid menu choice test
FR-11	Exit with thank-you message	Main menu loop	Exit test

SYSTEM ARCHITECTURE :

The application follows a simple layered idea. The main menu loop receives the user's choice and calls the matching operation function. The operation functions read from the shared **products** dictionary and read from and write to the shared **cart** list, and print their result back to the console.

Keeping each operation in its own function separates user interaction from business rules to a useful degree. For example, the discount slab logic lives only inside `generate_bill()`, so it can be changed in one place without touching the menu.

- User → Main menu loop
- Main menu loop → Operation function (display / add / view / bill)
- Operation function → products dictionary and cart list
- Result message → Console → User

USE CASE ANALYSIS :

The main actor is the Customer (or the cashier working for the customer). The actor interacts with the system through the terminal menu, and each menu option represents a use case. Every use case begins at the main menu and returns to it once the operation is complete.

- Display Products
- Add Product to Cart
- View Cart
- Generate Bill
- Exit

DETAILED WORKFLOW :

When the program starts, the products dictionary and an empty cart list are created and the menu is displayed inside a while True loop. The selected operation is processed and the result is displayed. The loop continues until the user selects the exit option.

For bill generation, the system first checks that the cart is not empty. It then adds up all line totals to get the subtotal, chooses the discount from the subtotal, subtracts the discount, calculates GST on the reduced amount and adds it to get the final amount.

- Start
- Create products dictionary and empty cart
- Display menu
- Read choice
- Process request
- Display result
- Return to menu
- Exit on option 5

SEQUENCE OF OPERATIONS :

A typical add-to-cart interaction begins with the user choosing option 2. The program displays the catalogue, reads a product number, checks it against the dictionary, reads a quantity, calculates the line total and appends the item to the cart.

- User → Menu (choice 2)
- Menu → add_to_cart()
- add_to_cart() → display_products()
- add_to_cart() → products dictionary (lookup of product number)
- add_to_cart() → line total = price × quantity
- add_to_cart() → cart list (append)
- Result → Console → User

MODULE AND COMPONENT DESIGN :

The program is divided into focused functions so that each has a clear, single responsibility. This makes the code easier to navigate, test and extend. The menu is numbered as follows: 1 Display products, 2 Add product to cart, 3 View cart, 4 Generate bill, 5 Exit.

Function	Purpose
display_products()	Prints the catalogue with product number, name and price.
add_to_cart()	Shows the catalogue, reads a product number and quantity, and appends the item to the cart.
show_cart()	Lists every cart line and the subtotal, or reports that the cart is empty.
generate_bill()	Totals the cart, applies the discount slab and GST, and prints the final bill.
Main menu loop	Displays the menu, reads the choice and calls the matching function.

Each function has a small, clear purpose, which keeps the code short and makes it straightforward to locate the logic responsible for any menu option.

DATA MODEL AND STORAGE DESIGN :

The catalogue is held in a global dictionary named **products**. The product number is the key and the value is a two-item list holding the product name and its price. A dictionary was chosen because it gives direct lookup by product number, and validating a product number is a single membership test.

The cart is a global list named **cart**. Each cart line is itself a list: [name, price, quantity, total]. The order of items is preserved, so the bill prints items in the order they were added.

```
products = {
    1: ["Rice", 60],
    2: ["wheat", 50],
    ...
    10: ["Coffee", 90]
}

cart = [] # after adding Rice x 5:
# cart = [["Rice", 60, 5, 300]]
```

The data lives only in memory while the program runs, so it is lost when the program closes. Adding file or database storage is listed under future enhancements.

INPUT VALIDATION AND ERROR HANDLING :

Validation prevents incorrect data from entering the cart. The rules that the current version enforces are listed below.

- A product number that is not in the catalogue is rejected with 'Invalid product number.'
- Viewing an empty cart shows 'Your cart is empty.'
- Generating a bill for an empty cart shows 'Cart is empty. Add products first.'
- Invalid menu choices show an error and the menu is displayed again.
- Line totals, discount and GST are always calculated by the program.

The current version does not use try/except blocks, so non-numeric input for the product number or quantity is not handled gracefully, and the quantity is not checked to be positive. This is recorded honestly as a limitation and planned as a future enhancement.

CORE ALGORITHM LOGIC :

This section describes, at a conceptual level, the logic behind the most important operations. The notation is illustrative pseudocode rather than literal source code.

Add to cart

```
function add_to_cart():
    display products
    read choice
    if choice in products:
        read quantity
        total = price * quantity
        append [name, price, quantity, total] to cart
    else:
        print "Invalid product number."
```

Generate bill

```
function generate_bill():
    if cart is empty: print message and return
    subtotal = sum of all line totals
    if subtotal >= 1000: discount = 10% of subtotal
    else if subtotal >= 500: discount = 5% of subtotal
    else: discount = 0
    after_discount = subtotal - discount
    gst = 5% of after_discount
    final_amount = after_discount + gst
    print bill
```

PROCESSING AND BILLING LOGIC :

The bill is calculated in a fixed order: subtotal, then discount, then GST on the discounted amount, then the final amount. The discount slabs are:

Subtotal	Discount
Below Rs. 500	No discount
Rs. 500 to below Rs. 1000	5% of subtotal
Rs. 1000 or more	10% of subtotal

A worked example for each slab, using the same inputs as the test table, is shown below.

Cart	Subtotal	Discount	After discount	GST (5%)	Final amount
Rice x5, Milk x2	Rs. 360	Rs. 0	Rs. 360	Rs. 18.0	Rs. 378.0
Shampoo x5	Rs. 600	Rs. 30.0 (5%)	Rs. 570	Rs. 28.5	Rs. 598.5
Shampoo x10	Rs. 1200	Rs. 120.0 (10%)	Rs. 1080	Rs. 54.0	Rs. 1134.0

The important point is that GST is charged on the amount *after* the discount, so the customer benefits from the discount on the tax as well. The discount slab is chosen from the subtotal before any discount.

IMPLEMENTATION AND PYTHON CONCEPTS :

The project applies functions, dictionaries, lists, loops, conditional statements and console input/output. These concepts work together to create a complete application rather than isolated code examples.

- Dictionary for the product catalogue
- List of lists for the cart
- Functions for each operation
- while True loop for the menu
- for loop to display products and cart items
- if / elif / else for menu selection and discount slabs
- Membership testing with the in operator
- Type conversion with int() for product number and quantity
- Arithmetic operators and the round() function for the bill
- Console input and output

Using these fundamentals together, rather than in isolation, was a deliberate choice: it mirrors how introductory concepts are combined in real, working software.

TESTING STRATEGY AND TEST CASES :

The program was run with a sequence of inputs covering the normal path, each discount slab and the error messages. The outputs were compared with hand-calculated values, and the results matched in every case.

Test	Input	Expected result	Status
Display products	Option 1	All 10 products with prices	Pass
Add product	Option 2, product 1, quantity 5	Product added to cart!	Pass
Invalid product	Option 2, product 99	Invalid product number.	Pass
View empty cart	Option 3 on a new session	Your cart is empty.	Pass
View cart	Rice x5, Milk x2, then option 3	Two lines, Subtotal Rs. 360	Pass
Bill on empty cart	Option 4 on a new session	Cart is empty. Add products first.	Pass
Bill, no discount	Rice x5, Milk x2, option 4	Discount 0, GST 18.0, Final 378.0	Pass
Bill, 5% discount	Shampoo x5 (Rs. 600), option 4	Discount 30.0, GST 28.5, Final 598.5	Pass
Bill, 10% discount	Shampoo x10 (Rs. 1200), option 4	Discount 120.0, GST 54.0, Final 1134.0	Pass
Invalid menu choice	Option 9	Invalid choice. Please try again.	Pass
Exit	Option 5	Thank you! Visit again., program ends	Pass

Cases known to fail (recorded honestly as limitations, not counted above): entering text such as *abc* for a product number or quantity stops the program with a `ValueError`, and a negative quantity is accepted and produces a negative line total.

SAMPLE OUTPUT :

The following console output was produced by running `shopping_billing_system.py` with the inputs listed in the test table.

Main menu and product list

```
=====
                SHOPPING & BILLING
=====
1. Display Products
2. Add Product to Cart
3. View Cart
4. Generate Bill
5. Exit
=====

===== PRODUCT LIST =====
1 . Rice - Rs. 60
2 . Wheat - Rs. 50
3 . Sugar - Rs. 45
4 . Milk - Rs. 30
5 . Biscuits - Rs. 25
6 . Chocolate - Rs. 40
7 . Soap - Rs. 35
8 . Shampoo - Rs. 120
9 . Juice - Rs. 50
10 . Coffee - Rs. 90
=====
```

Cart with two items (Rice x5, Milk x2)

```
===== YOUR CART =====
Rice | Price: 60 | Quantity: 5 | Total: 300
Milk | Price: 30 | Quantity: 2 | Total: 60
-----
Subtotal: Rs. 360
```

Bill without discount

```
=====
                FINAL BILL
=====
Rice x 5 = Rs. 300
Milk x 2 = Rs. 60
-----
Subtotal      : Rs. 360
Discount      : Rs. 0
GST (5%)      : Rs. 18.0
-----
Final Amount   : Rs. 378.0
=====
Thank you for shopping!
```

Bill with 5% discount (Shampoo x5)

```
=====
                FINAL BILL
=====
Shampoo x 5 = Rs. 600
-----
Subtotal      : Rs. 600
Discount      : Rs. 30.0
GST (5%)      : Rs. 28.5
-----
Final Amount   : Rs. 598.5
=====
Thank you for shopping!
```

Bill with 10% discount (Shampoo x10)

```
=====
                FINAL BILL
=====
Shampoo x 10 = Rs. 1200
-----
Subtotal      : Rs. 1200
Discount      : Rs. 120.0
GST (5%)      : Rs. 54.0
-----
Final Amount   : Rs. 1134.0
=====
Thank you for shopping!
```

Error messages

```
Enter your choice: 9
Invalid choice. Please try again.

Enter product number: 99
Invalid product number.

Enter your choice: 3
Your cart is empty.

Enter your choice: 4
Cart is empty. Add products first.
```

DESIGN DECISIONS :

The implementation choices were made to keep the project understandable while demonstrating clear organisation.

- One function per operation keeps responsibilities separate.
- A dictionary gives direct lookup of a product by its number.
- Each cart line stores the price and total at the time of adding, so the bill can be printed without looking up the catalogue again.
- The discount is decided from the subtotal and GST is applied afterwards, so the order of calculation is fixed and predictable.
- Menu choice is read as text and compared with strings, so a wrong menu entry can never crash the program.
- A while True menu loop keeps the program running until the user chooses to exit.
- A console interface avoids unnecessary external dependencies.

ASSUMPTIONS AND CONSTRAINTS :

A few simplifying assumptions were made to keep the project within the scope of a first-semester course, while still producing a usable tool.

- A single user runs the application on a single machine at a time.
- All prices are in a single currency (Rs.), without conversion.
- The product list and prices are fixed inside the program.
- The user enters product numbers and quantities as whole numbers.
- The program is run from a terminal with a standard Python 3 interpreter available.
- Data only needs to exist for the length of one session.

CHALLENGES AND SOLUTIONS :

Getting the billing order right was the main challenge. The discount depends on the subtotal, but GST must be charged on the discounted amount, so the calculation had to be done in a fixed sequence. The solution was to compute the subtotal first, choose the discount slab, and only then calculate GST.

Storing the cart in a form that is easy to print was solved by keeping each line as [name, price, quantity, total], so both the cart view and the bill simply loop over the list.

Keeping the program readable as features were added was handled by giving each operation its own function instead of writing everything inside the menu loop.

PROJECT TIMELINE AND WORK BREAKDOWN :

The project was carried out in stages, moving from requirements to a working, tested application. The approximate breakdown of effort is shown below.

Stage	Activity
1	Requirements gathering and problem definition
2	Data structure and function design
3	Core implementation (product list, add to cart, view cart)
4	Billing logic (discount slabs and GST)
5	Menu integration and manual testing
6	Test execution and result recording
7	Report documentation

LEARNINGS AND OUTCOMES :

The project provides practical experience in converting a problem into a structured Python application and connecting user input, processing and output.

- Functions and program structure
- Dictionaries and lists, including nested lists
- Loops and conditional logic
- Percentage, discount and tax calculations in code
- Manual test design
- Project documentation

LIMITATIONS AND FUTURE ENHANCEMENTS :

Limitations

- Data is not persistent; it is lost when the program closes.
- No exception handling: non-numeric product number or quantity stops the program.
- Quantity is not checked, so zero or negative quantities are accepted.
- Items cannot be removed from the cart, and the cart is not cleared after a bill is generated.
- Adding the same product twice creates two separate cart lines.
- The product list is fixed in the code.
- Floating-point numbers are used for money.

Future enhancements

- Add try/except blocks and reject quantities below one.
- Add options to remove an item and to clear the cart after billing.
- Merge repeated products into a single cart line.
- Save bills to a file (text, JSON or CSV) and load the catalogue from a file.

- Add customer details, a bill number and the date and time on the bill.
- Add stock management and payment options.
- Use classes (OOP) and the decimal module for accuracy.
- Build a GUI (Tkinter) or web interface.

CONCLUSION :

The Shopping and Billing System demonstrates how an everyday problem can be addressed with a structured Python application. It combines menu-driven input handling, dictionary-based product storage, a list-based cart and automatic discount and GST calculation to implement the core steps of a shop billing counter.

The current version provides a clear foundation that can be extended with input validation, persistence and a richer interface, while remaining suitable for an introductory academic project.

GLOSSARY OF TERMS :

Term	Meaning
Product number	The number used as the dictionary key to select a product.
Cart	The list that holds the items the customer has chosen.
Quantity	The number of units of a product added to the cart.
Subtotal	The sum of all line totals before discount and tax.
Discount	A percentage reduction of the subtotal, decided by the discount slab.
GST	Goods and Services Tax, charged here at 5% of the discounted amount.
Final amount	The amount payable: subtotal minus discount plus GST.
Dictionary	A Python data structure that stores key-value pairs.
CLI	Command-Line Interface; a text-based way of using a program.

USER MANUAL – HOW TO RUN THE APPLICATION :

The following steps describe how to run the Shopping and Billing System from a terminal.

- 1. Install Python 3 if it is not already available on the system.
- 2. Open a terminal and navigate to the project folder.
- 3. Run the application with: `python shopping_billing_system.py`
- 4. Choose an option from the numbered menu (1 to 5).
- 5. To buy items: choose option 2, enter the product number, then the quantity. Repeat for more items.
- 6. Choose option 3 to view the cart and option 4 to generate the bill.
- 7. Select option 5 to exit the program.

REFERENCES AND SUBMISSION CHECKLIST :

The implementation uses core Python concepts and course learning context. Before submission, all project details and runtime behaviour should be verified.

- Python documentation: docs.python.org
- CSE 1021 course learning materials
- Student-created implementation and test cases
- Run every menu option
- Verify student details and date
- Check the final ZIP contents